Jacob Beal · CS 6050 · Graduate Algorithm Project 2

Knuth–Plass optimum-fit line breaking

Source: D. E. Knuth & M. F. Plass, "Breaking Paragraphs into Lines," *Software: Practice and Experience* 11(11), 1981, translated from its pseudocode. Code: `knuth_plass/knuth_plass.py`. Theory (p. 1148): O(B·W) for B legal breakpoints, W words per line.

The problem and the recurrence

Problem: find the breaks with the fewest total demerits. The line a→b has ratio $r_{ab} = (l - L)/Y$ if shorter than the width l, $(l - L)/Z$ if longer (L = natural width, Y = stretch, Z = shrink), fits if $-1 \le r_{ab} \le \rho$, and has class 0 tight ($r < -0.5$), 1 normal, 2 loose ($r > 0.5$) or 3 very loose ($r > 1$).

State: D[b, c] = fewest total demerits to set the text through legal breakpoint b, with the line ending at b in class c. P[b, c] = the minimizing (a, c′).

Base: D[0, 1] = 0 (the start of the paragraph acts like the end of a normal line, and $f_0 = 0$)

Recurrence: D[b, c] = min over (a, c′) of **D[a, c′] + δ(a, b) + α·f_a·f_b + γ·[|c − c′| > 1]**, over every legal breakpoint a < b (or a = 0) and class c′ with D[a, c′] < ∞, where the line a→b fits, has class c and passes no forced break. D[b, c] = ∞ if there is none.

Cost: $\delta(a, b) = (1 + 100|r_{ab}|^3 + p_b)^2$ if $p_b \ge 0$, $(1 + 100|r_{ab}|^3)^2 - p_b^2$ if $p_b < 0$, $(1 + 100|r_{ab}|^3)^2$ at a forced break; $f_b = 1$ at hyphens.

Answer: min over c of D[m, c], where m is the forced break that ends the paragraph. Following P back to 0 gives the breaks.

Why every possible final line is covered. In an optimal solution $b_1 < \ldots < b_k = m$, the last line runs from $a = b_{k-1}$ (or 0) to m in class c, after a line of class c′. a is a legal breakpoint before m and a→m fits, so (a, c′) is a candidate. The last line's cost depends only on a, m, c, f_a and c′, so the prefix matters only through its cost, which is at least D[a, c′]: that candidate is ≤ the optimum, and every candidate is a real setting's cost, so the minimum is the optimum. By induction on b, every D[b, c] is correct. c′ must be in the state because γ depends on it (dropping classes gave worse answers on 40 of 148,000 random paragraphs). My code skips only candidates that can't win: an a whose line is already too long, since it only gets longer, and a class more than γ above the cheapest one at a, since a class changes later costs by at most γ.

Worked example

"when she was bored she took a golden ball, and threw it up on" (word widths 41, 25, 29, 44, 25, 35, 9, 51, 34, 29, 45, 12, 20, 19), l = 236, spaces (6, 3, 2), ρ = 1, γ = 3000, and no hyphens, so α never applies. 3 of its $2^{13} = 8{,}192$ sets of breaks fit. For example, "when she was bored she took" has L = 199 + 5·6 = 229, Y = 5·3 = 15, r = 7/15 = 0.467 (class 1) and $\delta = (1 + 100 \cdot 0.467^3)^2 = 124.61$. No line ending after the other 10 words fits, so D = ∞ there. Every candidate for the other entries, in fill order, from `knuth_plass/worked_example.py`:

Entry	From (a, c′)	Line a→b	r	c	D[a, c′] + δ + γ = candidate	Minimum ← predecessor
D[took, 1]	(start, 1)	when she was bored she took	+0.467	1	0 + 124.61 = 124.61	124.61 ← (start, 1)
D[a, 0]	(start, 1)	when she was bored she took a	−0.667	0	0 + 938.17 = 938.17	938.17 ← (start, 1)
D[up, 1]	(took, 1)	a golden ball, and threw it up	0.000	1	124.61 + 1.00 = 125.61	125.61 ← (took, 1)
D[up, 2]	(a, 0)	golden ball, and threw it up	+1.000	2	938.17 + 10,201.00 + 3,000 = 14,139.17	14,139.17 ← (a, 0)
D[on, 0]	(a, 0)	golden ball, and threw it up on	−0.833	0	938.17 + 3,465.72 = 4,403.89	4,403.89 ← (a, 0)
D[on, 1]	(up, 1)	on	+0.002	1	125.61 + 1.00 = 126.61	**min = 126.61 ← (up, 1)**
	(up, 2)	on	+0.002	1	14,139.17 + 1.00 = 14,140.17	loses to (up, 1)

Checked but not candidates, because the line doesn't fit: took→a (r = +∞), start→up (r = −9.79), start→on (r = −10.0), took→on (r = −1.79).

The answer is min(D[on, 0], D[on, 1]) = min(4,403.89, 126.61) = **126.61**. Tracing P back, (on, 1) ← (up, 1) ← (took, 1) ← (start, 1) gives "when she was bored she took | a golden ball, and threw it up | on", matching my code and a brute-force search.

What I learned

- *Contiguous* lines make line breaking a shortest path, not bin packing.
- The cheapest way to a break can lose, so the state needs the class.
- Tests can pass and still miss bugs: mine missed 2 of 10 that I planted.

How I created the test cases

`knuth_plass/generate_tests.py` (seeded) writes 10 paragraphs that double in size, from 2,048 to 1,048,576 words (2,353 to 1,216,218 breakpoints), sampled from the paper's Grimm paragraph, with a 500-unit column (W ≈ 12.3), ρ = 1.26 and α = γ = 3000. Expected demerits come from a separate backward DP, and a case passes if they match and rescoring its breaks agrees. Times are the minimum of at least 3 runs, garbage collector off, on an M1 Max with Python 3.12. Unit tests also match the paper's 21 published breaks and brute force. The paper omits its font table, so some letter widths are estimates and demerit totals differ from its tables by 9% and 16%.

Runtime vs. theory

With W fixed the bound predicts linear growth in B. In Figure 1 (top) the log–log slope is **1.008** over the five largest cases and 1.017 over all ten, against 1.079 for B log B, and each doubling took 1.97–2.09× as long. The smallest cases run about 10% low: opening lines have few feasible breaks (p. 1163), so the first 250 words average 11.0 active entries per breakpoint against 18.3 later, and each entry costs 281 ns against 300 ns. For W (bottom) the slope is **1.054** once W reaches 45 but 1.193 overall: active entries per W rise from 1.5 to 3.5, then level off, since long lines let each break keep more of its 4 class entries (a constant hidden in the O). With γ = 0 the ratio stays 1.2–1.3 (slope 0.928). I believe the implementation matches O(B·W).

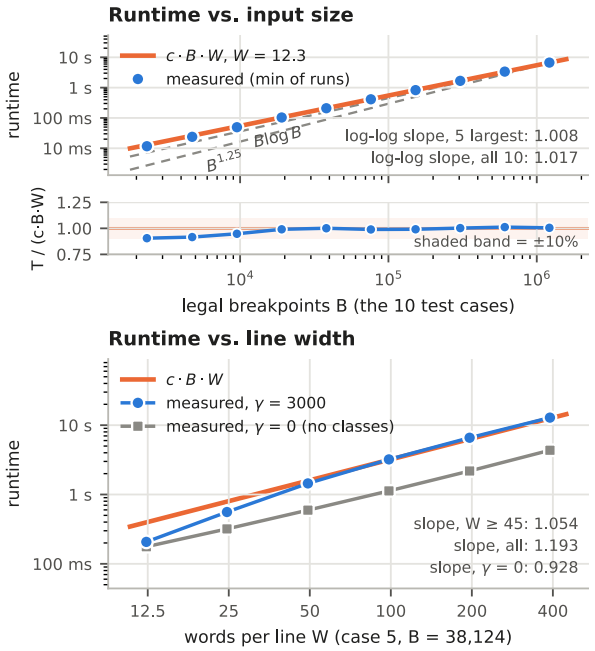


Figure 1. Top: runtime vs. B, the fitted c·B·W line (orange), other growth rates (gray) and runtime ÷ fit. Bottom: runtime vs. W with B fixed, c fit where W ≥ 45.